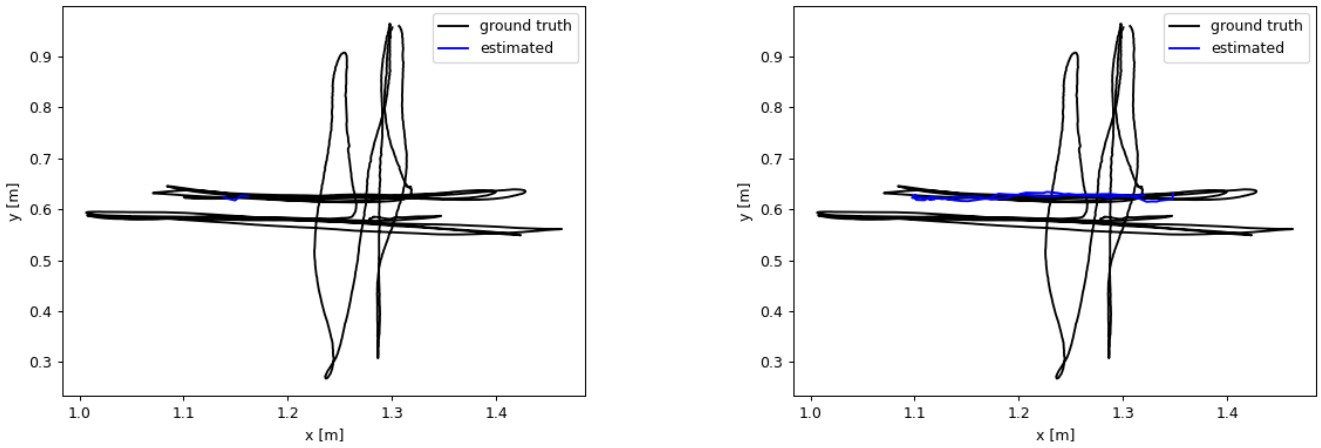
Lab 4: Visual SLAM - Continuation

Team 8: Piyush Tiwari, Diego Méndez Carter March 30, 2026

Disclaimer: Laboratory 3 : Durign the last lab, the results obtained , presented differences from the expected output. This was nothing but an error in the configuration file. Not due an error in the implemented logic. For demonstrating the point, the corrected tracking maps can be found on the following figure as proof of this tweak before starting this session.



(a) Lab 3 session: tracking lost immediately due to misconfiguration.

(b) Corrected: estimated trajectory follows ground truth before losing track.

Figure 1: Comparison before and after fixing `Map.minObs` in `TUMRGBD.yaml`.

With this fix confirmed, we proceed to complete Laboratory 4. The present work is divided into the story of how we built each task and gave stability to the system until we obtained the very first complete trajectory on the *freiburg xyz* dataset. After achieving this, we were able to start experimenting and tuning the parameters on the *vicon room* dataset.

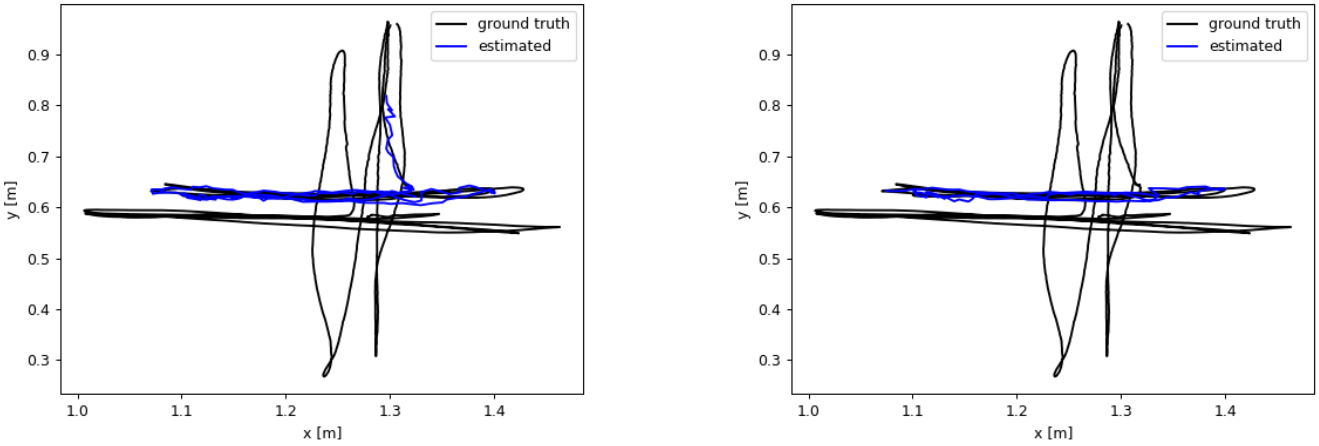
1 KeyFrame Insertion

A frame that carries novel visual information for the map is called a **KeyFrame**. The Local Mapping thread processes KeyFrames to triangulate new MapPoints 3D points in the real world and refine the map itself. Inserting KeyFrames in a generous and fast way is the path for making the camera not lose track and letting the system eliminate redundancy later. As stated in the ORB-SLAM paper [1], the system tries to insert KeyFrames as fast as possible because it makes tracking more robust to challenging camera movements, typically rotations. We should insert a KeyFrame when the camera sees enough new information or when it is almost going to get lost. For this, the system should be capable of detecting:

- **Time passing:** A minimum number of frames has passed since the last KeyFrame was inserted.
- **Weakness:** The camera is tracking a low number of points, so a new KeyFrame must be inserted immediately.
- **Visual change:** The current frame cannot track 90% of the points that the reference KeyFrame can observe, indicating sufficient camera displacement.

- **Availability:** The Local Mapping thread is inactive and ready to receive new information.

The insertion policy follows the concept of *survival of the fittest* described in [1] — the system is generous when creating KeyFrames and MapPoints to boost tracking robustness, while a later restrictive culling mechanism discards redundant ones.



(a) One run with different parameters

(b) With KeyFrame insertion in different run

Figure 2: Estimated trajectory (blue) vs. ground truth (black) before and after Task 1.

For this task, the selected values were the following:

- **minFrames:** Minimum number of frames between two consecutive KeyFrame insertions.
- **nFeatTracked:** If the system tracks fewer points than this threshold, a new KeyFrame is inserted urgently.
- **Timeout (nFramesFromLastKF):** Maximum number of frames without insertion. If this limit is exceeded, a KeyFrame is inserted even if there is no significant new visual information.
- **Ratio:** A KeyFrame is inserted if the current frame tracks less than this percentage of points with respect to the reference KeyFrame.

Table 1: KeyFrame insertion strategies evaluated on *freiburg1_xyz*. Strategy 5 achieved the highest number of KeyFrames inserted before crash.

Strategy	minFrames	nFeatTracked	Timeout	Ratio (%)	KFs inserted
1	15	40	30	90	7
2	20	50	35	95	6
3	5	40	15	90	5
4	15	50	20	90	7
5	15	45	25	90	**8**

The insertion pattern observed with Strategy 5 was the following:

- KeyFrames 1, 2, 5, 7 and 8 were inserted by the **visual change** condition — the current frame tracked less than 90% of the points observed by the reference KeyFrame, indicating sufficient camera displacement.
- KeyFrames 3, 4 and 6 were inserted by the **timeout** condition — a maximum number of frames had elapsed without insertion, ensuring the map is updated even during gradual motion.

The system crashed at KeyFrame 8 due to the accumulation of invalid MapPoints, which can be fixed by adding the culling and the fusing strategies tigether. The trajectory estimated up to that point achieved an **ATE RMSE of** 1.1 **cm** on the *freiburg1_xyz* sequence. This result suggests that the more balanced insertion policy of Strategy 5 provides better pose estimates by maintaining a more consistent local map during the short window before the system crashes. An example of this behaviour is shown in Fig. 3.



Figure 3: System inserting KeyFrames on *freiburg1_xyz*.

2 New MapPoints Triangulation

New MapPoints are triangulated by matching the current KeyFrame with its covisible neighbors in the map. As described in [1], each candidate point must pass the following geometric checks before being added to the map:

1. **Sufficient parallax:** the cosine of the angle between the two viewing rays must satisfy $\cos(\alpha) < 0.9998$. Points triangulated from nearly identical viewpoints are geometrically unstable and are discarded.
2. **Positive depth:** the triangulated point must lie in front of both cameras ($Z_1 > 0$, $Z_2 > 0$). Points behind a camera correspond to physically impossible observations.
3. **Low reprojection error:** the point is reprojected into both frames and must satisfy $e^2 < 5.991 \cdot \sigma^2$, where σ is the scale uncertainty at the detected keypoint level. This threshold corresponds to the χ^2 test at 95% confidence with 2 degrees of freedom, and rejects points inconsistent with the estimated geometry.

Points passing all checks are inserted into the global map and registered as observations from both KeyFrames.

Compared with the results shown in the Disclaimer (Figure 1b), where the estimated trajectory collapsed into a near-stationary segment, the improvement is evident: the system now continuously expands its map as new KeyFrames are inserted, triangulating fresh MapPoints that sustain tracking across the full camera trajectory. However, the system still loses track easily due to the absence of a MapPoint culling mechanism, which will be addressed in the following task.

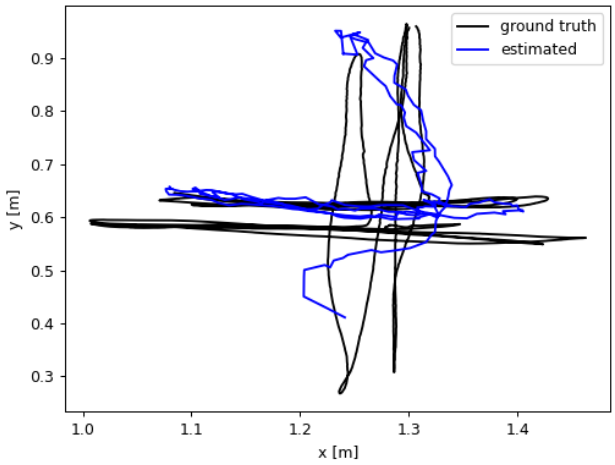


Figure 4: Estimated trajectory (blue) vs. ground truth (black) after implementing `triangulateNewMapPoints()` on *freiburg1_xyz*.

3 MapPoint Fusion and Culling

Once new MapPoints are triangulated, two complementary mechanisms maintain map quality as described in [1]: fusion merges duplicate points, while culling removes unreliable ones..

3.1 MapPoint Fusion

After triangulating new MapPoints for each KeyFrame, the map may contain duplicate points — two distinct 3D points representing the same physical location in the world. To address this, each MapPoint is projected into neighboring KeyFrames and matched against their keypoints using descriptor similarity. This process is conceptually similar to the `searchWithProjection` function implemented in Laboratory 3, where MapPoints from the local map are projected into the current frame to find new matches.

After finding a candidate match, two cases are possible. If the matched keypoint has no MapPoint associated (Case A), the observation is simply added to the existing MapPoint. If the matched keypoint already has a different MapPoint associated (Case B), both points are fused into a single one, keeping the one with more observations and transferring all remaining observations to it. This reduces redundancy in the map and improves the robustness of the data association.

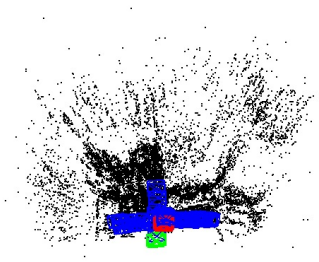
3.2 MapPoint Culling

The MapPoint culling strategy aims to eliminate spurious points that introduce noise and degrade the precision of the system. Incorrect triangulations and false matches produce MapPoints that are never reliably tracked, increasing the Bundle Adjustment cost without contributing to map accuracy. As stated in [1], the goal is to maintain a compact and accurate map that remains bounded in size even during continuous operation in the same environment.

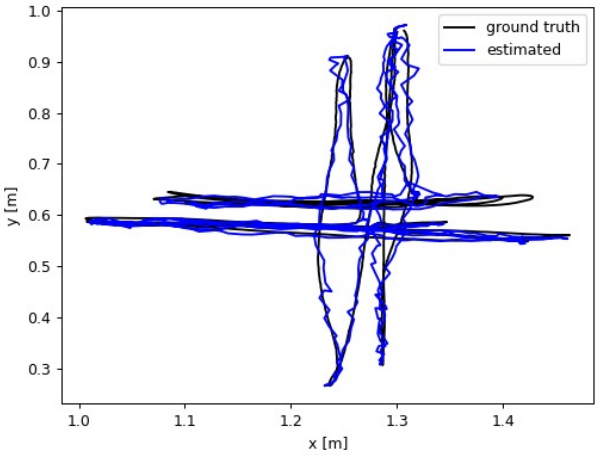
Following the criteria described in [1], a MapPoint is considered unreliable and removed if any of the following conditions are met:

- **Visibility test:** the point must be tracked in more than 25% of the frames where it is predicted to be visible. Points that consistently fail to be matched are considered spurious.
- **Consensus of observations:** after more than one KeyFrame has passed since its creation, the point must have been observed from at least 3 KeyFrames. Points with fewer observations provide insufficient constraints for the Bundle Adjustment.
- **Permanent degradation:** a point is permanently removed if its number of observations drops below 3 at any time after passing the initial test, which can happen when KeyFrames are culled or when the Bundle Adjustment rejects it as an outlier.

In our implementation, culling is only activated after at least 3 KeyFrames have been inserted into the map, to avoid eliminating valid points that have not yet had time to accumulate observations. This policy guarantees the precision and quality of the map while keeping the system stable during the early stages of exploration.



(a) Dense reconstruction during tuning of the final tasks.



(b) Complete execution of the *freiburg1_xyz* dataset.

Figure 5: Full dataset completed after implementing all mapping tasks.

After completing the *freiburg1_xyz* trajectory, we proceeded to tune and evaluate the system on the *vicon_room* dataset.

4 Experiment over the Vicon dataset.

The *V1_01_easy*, *V1_02_medium*, and *V1_03_difficult* sequences from the EuRoC MAV benchmark present progressively faster camera motions and more challenging scenarios for feature extraction compared to the TUM RGB-D dataset. In each of the following sections, we evaluate the impact of the relevant parameters on system performance by varying one parameter at a time while keeping the others fixed, analyzing the resulting trends in accuracy and robustness as the parameter values change.

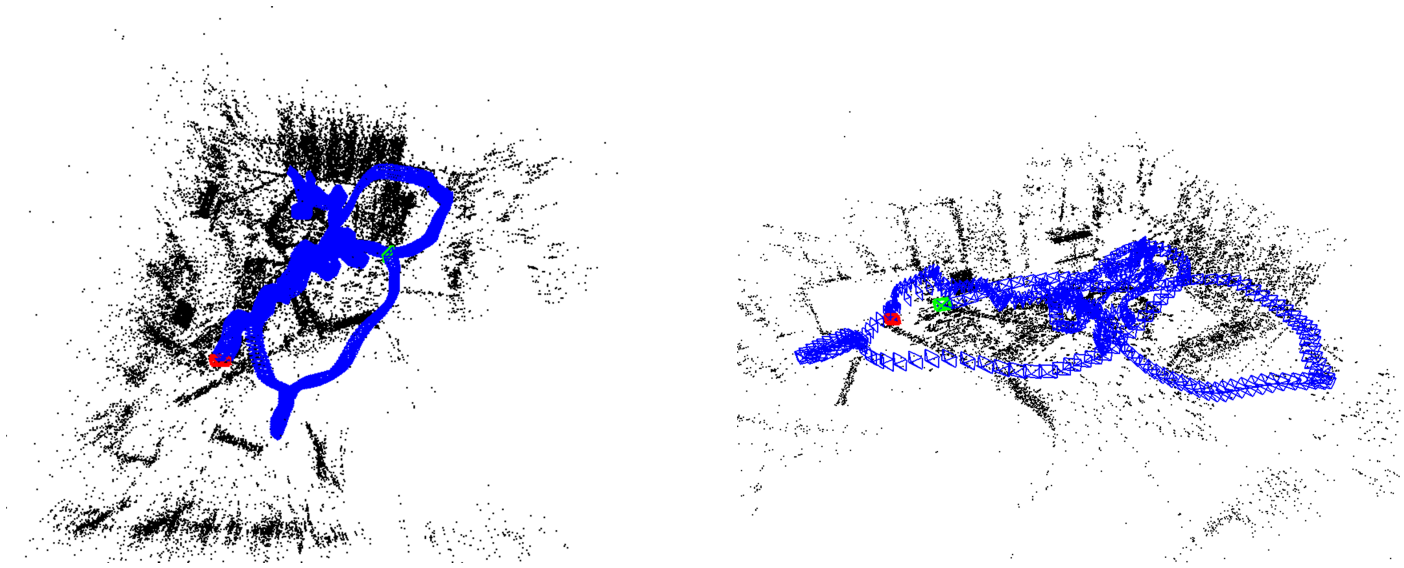
4.1 Keyframe strategy

4.1.1 Easy

In Table 2, the results are consistent with the KeyFrame insertion philosophy of doing it as fast as possible to ensure tracking robustness. Experiments 3 and 4, using restrictive parameters (`minFrames`= 15 and `nFeatTracked`= 30, respectively), fail to maintain tracking due to the lack of sufficient KeyFrames to fulfill the needs of the system. Experiment 2 with `minFrames`= 2 inserts 855 KeyFrames, far more than necessary, without improving accuracy. Experiments 1 and 5 achieve a balance between insertion frequency and map quality, with Experiment 1 yielding the lowest ATE RMSE at 8.88 cm.

Table 2: KeyFrame insertion strategies evaluated on *V1_01_easy*. The best ATE was achieved by Experiment 5.

Exp.	minFrames	nFeatTracked	Timeout	Ratio (%)	KFs	ATE RMSE (cm)	Status
1	5	65	15	90		8.88	✓
2	2	65	8	90	855	9.30	✓
3	15	30	25	85	15	—	Lost
4	10	45	15	85	74	—	Lost
5	10	65	15	85	251	**9.22**	✓



(a) Dense map

(b) Non-saturated map

Figure 6: Comparison of map density on *V1_01_easy*.

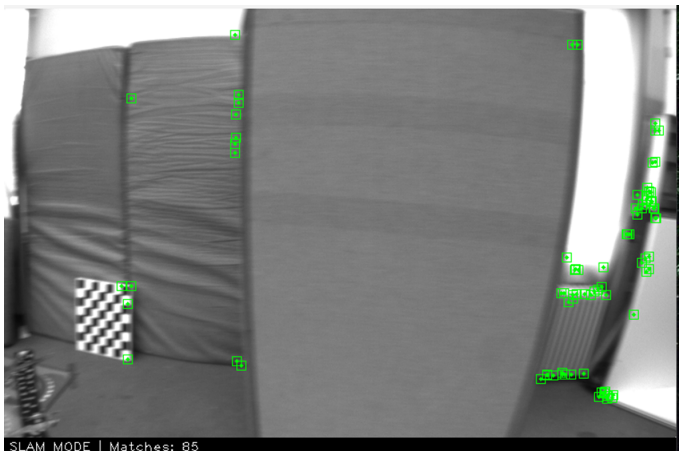
4.1.2 Medium

In Table 3, all experiments on *V1_02_medium* either fail entirely or lose tracking at the same point in the sequence, where the camera faces a textureless column (Figure 11). Experiments 1, 3, and 5 fail with both conservative and overly aggressive parameters, while Experiments 2 and 4 reach the same region before losing tracking with ATE RMSE of 2.58 cm and 2.90 cm respectively. This confirms that the tracking loss is not caused by the KeyFrame insertion policy but by an inherent limitation of the ORB detector on textureless surfaces.

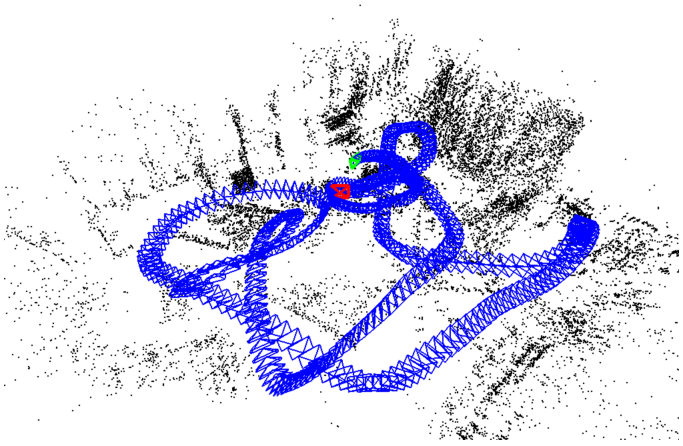
Table 3: KeyFrame insertion strategies evaluated on *V1_02_medium*. All successful experiments lose tracking in the same low-texture region.

Exp.	minFrames	nFeatTracked	Timeout	Ratio (%)	KFs	ATE RMSE (cm)	Status
1	5	65	15	90	—	—	Lost
2	2	65	8	85	355	9.99	Partial
3	5	45	12	85	52	—	Lost
4	3	65	10	80	238	**9.97**	Partial
5	1	65	5	75	25	—	Lost

Figure 11 shows the frame where the system loses tracking and the best map reconstruction achieved with this configuration.



(a) Last tracked frame: features concentrate on edges with no keypoints on the textureless divider.



(b) Partial map reconstruction before tracking loss.

Figure 7: Tracking loss on *V1_02_medium* caused by a textureless surface in the environment.

4.1.3 Hard

In the middle of our impolmenetations, we explored an adaptive keyframe insertion experiment. Thinking that we could extend the tracking through challenging regions The idea was to dynamically adjust the `minFrames` cooldown based on tracking quality: when the number of tracked features drops below a threshold, the system accelerates KeyFrame insertion to capture the remaining visual information before tracking is lost.

Three adaptive variants were tested. Experiment 2 used an aggressive burst mode that eliminated the cooldown entirely when features dropped below 40, but it activated too late — by the time the burst

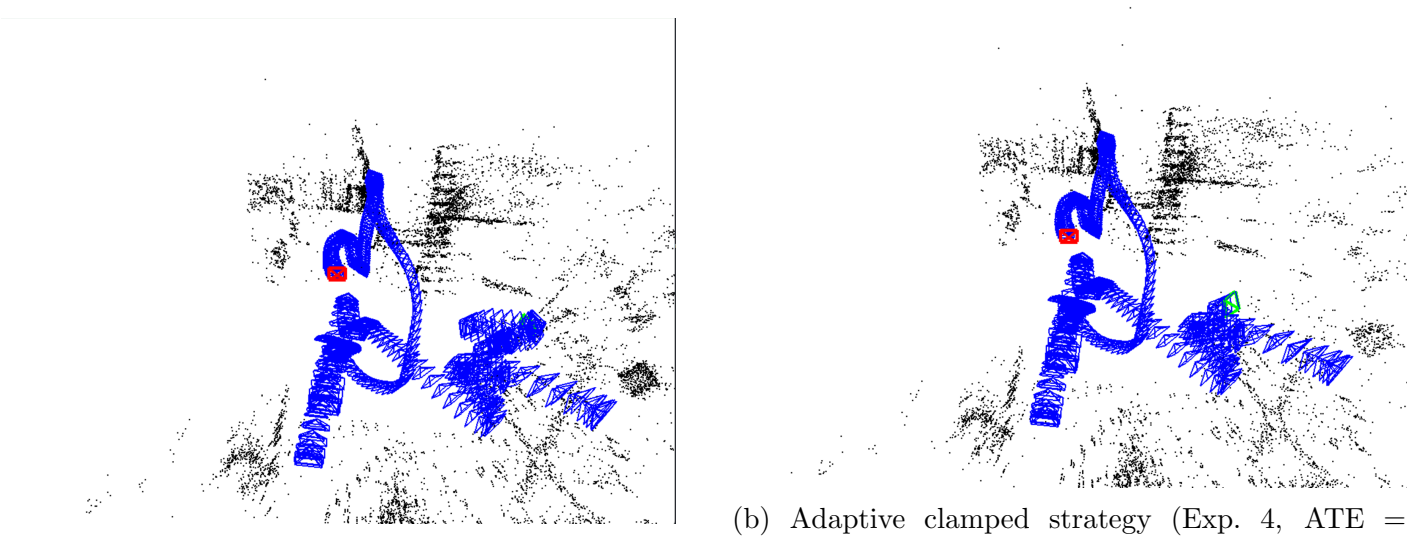
triggered, the map had already degraded from 25,000 to 2,400 edges. Experiment 3 used a smoother adaptation with a wider range (0–3), but the system relaxed too much during stable periods and could not recover quickly enough, losing tracking after only 60 KFs. Experiment 4 clamped the adaptive range to 1–2, matching the proven baseline as an upper bound. This achieved comparable coverage (352 KFs) but with degraded accuracy (12.05 cm vs 4.09 cm), as the additional KeyFrames inserted in low-feature regions introduced poorly constrained observations into the Bundle Adjustment.

These results suggest that adaptive insertion cannot overcome the fundamental limitations of the ORB detector in textureless or overexposed environments. The fixed configuration from Experiment 1 remains the best trade-off between accuracy and robustness for this sequence.

Table 4: KeyFrame insertion strategies evaluated on *V1_03_difficult*. Tracking loss is caused by overexposed regions and fast camera motion.

Exp.	**minFrames**	**nFeatTracked**	**Timeout**	**Ratio (%)**	**Adaptive**	**KFs**	**ATE RMSE (cm)**	**S**
1	2	65	8	85	No	351	4.09	P
2	adapt(0–3)	65	8	85	Burst	101	—	
3	adapt(0–3)	65	8	85	Smooth	60	—	
4	adapt(1–2)	65	8	85	Clamped	352	12.05	P

As shown in Figure 8, both strategies achieve similar trajectory coverage. However, the adaptive approach (Exp. 4) reaches the same point with slightly more KeyFrames (352 vs 351) while yielding a higher ATE RMSE (12.05 cm vs 4.09 cm), indicating that the additional KeyFrames inserted during low-feature periods degrade the overall map quality rather than improving it.



(a) Fixed parameters (Exp. 1, ATE = 4.09 cm).

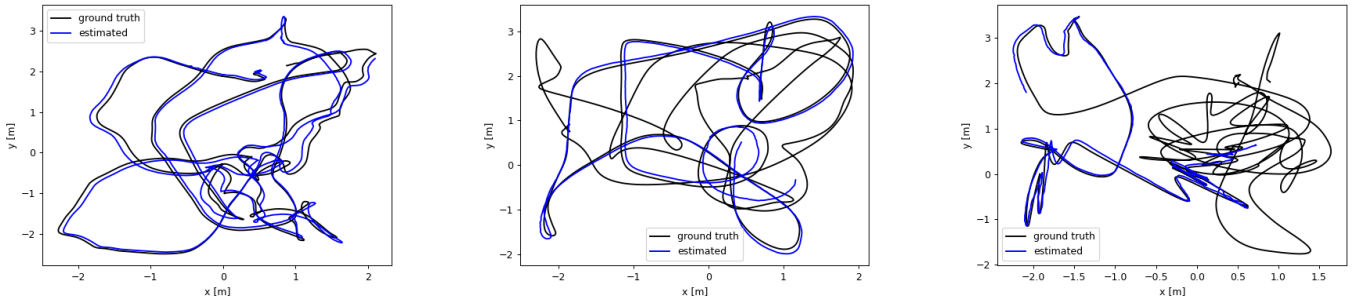
(b) Adaptive clamped strategy (Exp. 4, ATE = 12.05 cm).

Figure 8: Map reconstructions on *V1_03_difficult*: fixed vs adaptive insertion.

4.1.4 Conclusions after key insertion

Inserting KeyFrames too frequently saturates the Bundle Adjustment without improving accuracy, while being too conservative causes the system to lose tracking before it can react. The configuration `minFrames`= 3, `nFeatTracked`= 65, `Timeout`= 10, and `Ratio`= 80% offered the best balance across

the three sequences. That said, no single set of parameters works equally well everywhere — each sequence brings its own difficulties, from the textureless divider in medium to the overexposed window in difficult, and these are challenges that no insertion policy can fully resolve. Even the adaptive strategies we explored could not extend tracking beyond these limits, reinforcing that the bottleneck lies in the scene and the sensor rather than in the KeyFrame selection logic.



(a) *V1_01_easy* (b) *V1_02_medium* (c) *V1_03_difficult*

Figure 9: Best map reconstructions achieved on each EuRoC Vicon Room 1 sequence with the selected KeyFrame insertion configuration.

4.2 Fusion and culling experiments

Building on the implementation described in Sections 3, we evaluate three configurations that vary the culling strictness and fusion threshold simultaneously, from permissive to strict. The KeyFrame insertion policy remains fixed at the baseline selected in the previous section.

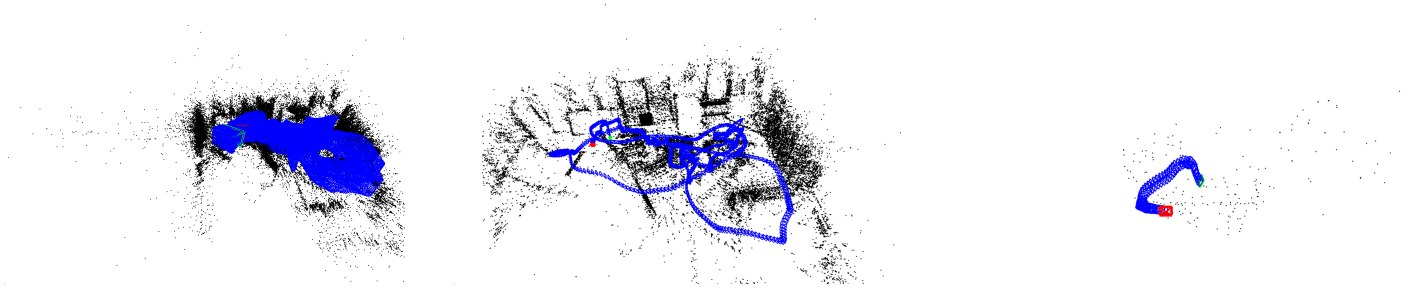
4.2.1 Easy Map

Table **??** evaluates three configurations ranging from permissive to strict, keeping the KeyFrame insertion policy fixed at the baseline selected in the previous section.

Table 5: Fusion and culling configurations evaluated on *V1_01_easy*. KeyFrame insertion is fixed at the baseline configuration.

Exp.	Fuse th	nObs	KF guard	KFs	ATE RMSE (cm)	Status
1	70	< 2	3	691+	32.43	Partial (slow)
2	50	< 3	5	893	9.78	✓
3	30	< 5	7	48	**1.89**	Partial (strict)

The permissive configuration (Exp. 1) retains almost every triangulated point, producing a dense but noisy map with over 691 KeyFrames. The Bundle Adjustment struggled to converge under this volume of low-quality observations, taking over 2 seconds per iteration and yielding the worst ATE of 32.43 cm. The strict configuration (Exp. 3) sits at the opposite extreme: by requiring at least 5 observations to keep a point, it aggressively prunes the map and achieves an excellent ATE of 1.89 cm — but only over 48 KeyFrames before running out of points to track. The baseline (Exp. 2) is the only configuration that completes the full sequence, balancing a reasonably clean map with enough points to sustain tracking throughout, at 9.78 cm.



(a) Permissive (32.43 cm) (b) Baseline (9.78 cm) (c) Strict (1.89 cm, 48 KFs)

Figure 10: Map reconstructions on *V1_01_easy* for each culling and fusion configuration.

4.2.2 Medium and Hard maps

The experiments on medium and difficult were run only with the baseline configuration to confirm its robustness across all three sequences, since the environmental limitations observed in these datasets — textureless surfaces and sensor overexposure, dominate over the culling and fusion parameters.

Table 6: Fusion and culling baseline evaluated on *V1_02_medium* and *V1_03_difficult*. KeyFrame insertion fixed at baseline.

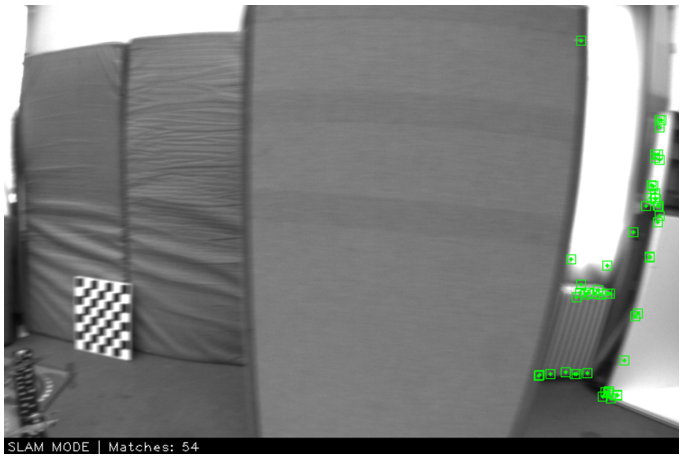
Seq.	Exp.	Fuse th	nObs	KF guard	KFs	ATE RMSE (cm)	Status
Medium	1	50	< 3	5	238	9.74	Partial
Hard	1	50	< 3	5	60	1.46	Partial
Hard	2	60	< 3	5	20	33.03	Lost

On *V1_02_medium*, the baseline configuration loses tracking at the same textureless divider observed in the KeyFrame insertion experiments (Figure 11), reinforcing that this failure is environmental and independent of the culling parameters. On *V1_03_difficult*, the picture is similar but with a different cause: the baseline achieves a precise yet short trajectory (1.46 cm, 60 KFs), while increasing the fuse threshold to 60 worsened results dramatically — only 20 KeyFrames with an ATE of 33.03 cm. Figure 12 illustrates both the resulting maps and the frames where tracking is lost: features collapse into a single cluster on the calibration pattern while the surrounding surfaces remain either textureless or overexposed, leaving the system with no spatial distribution to estimate the pose.

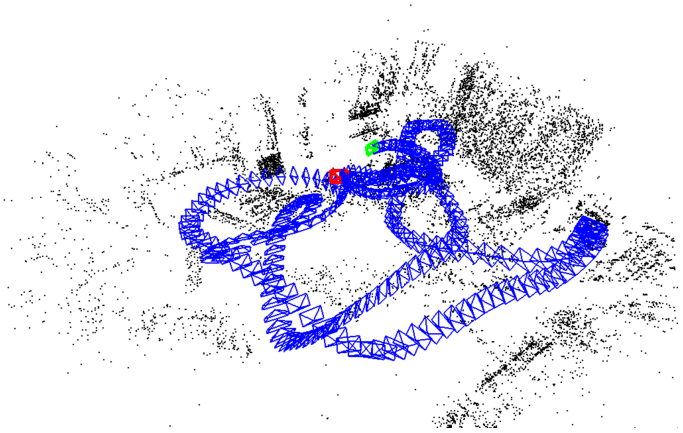
Across all three sequences, the culling and fusion experiments confirm that the baseline configuration (`Matching.fuse`= 50, `nObs`< 3, `KF guard`= 5) offers the best compromise between map cleanliness and trajectory coverage. Being too permissive floods the map with poorly constrained points that slow down the Bundle Adjustment and degrade accuracy, while being too strict starves the system of the points it needs to keep tracking. More importantly, these experiments reinforce a recurring observation: once the scene presents textureless surfaces or overexposed regions, no combination of culling or fusion parameters can compensate for the lack of visual information. The system's robustness is ultimately bounded by what the sensor and the ORB detector can extract from the environment.

5 Conclusions

Throughout the lessons and the experiments, we kept arriving at the same realization: tuning a visual SLAM system is difficult, and in the end it is not so much about finding the right numbers as it is about



(a) Last tracked frame with baseline culling.



(b) Partial map reconstruction before tracking loss.

Figure 11: Tracking loss on *V1_02_medium* on the same point due to texturless frame.

understanding what each choice trades away. Inserting KeyFrames too often does not make the map better, it makes the Bundle Adjustment heavier, since its cost depends on how many KeyFrames exist and how many features each one matches. At some point, adding more data stops helping and starts hurting.

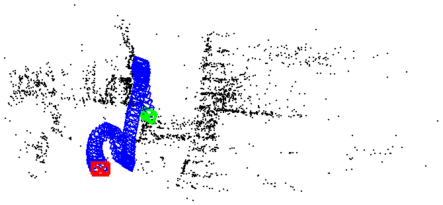
But perhaps the most honest conclusion is that there are situations where no amount of parameter tuning will help. When the camera faces a blank wall or a saturated window, the ORB detector simply has nothing to work with, and the system will lose tracking regardless of how carefully the insertion, culling, or fusion policies are configured. In our view, a monocular SLAM system can only ever be as good as what the camera is able to see.

Team Contributions

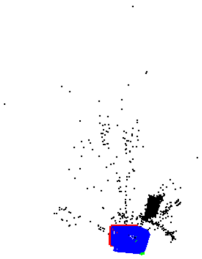
Diego implemented the base code, while Piyush stabilized some functions following the last lab session. Both we performed the experiments and the report for this part of the course.

References

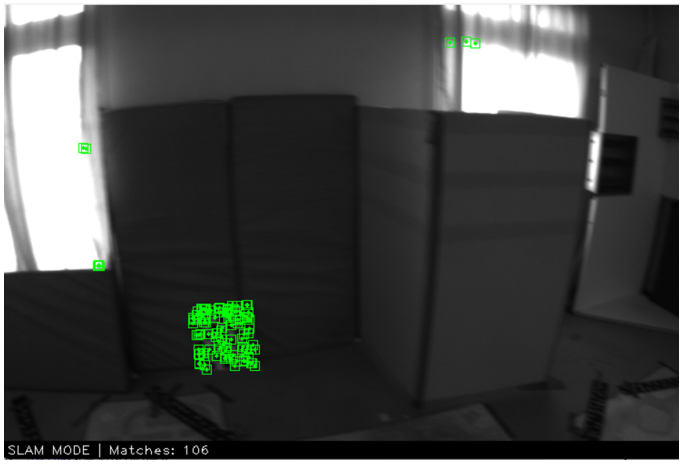
[1] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: A versatile and accurate monocular SLAM system," *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015.



(a) Baseline map (Exp. 1, ATE = 1.46 cm, 60 KFs).



(b) Fuse=60 map (Exp. 2, ATE = 33.03 cm, 20 KFs).



(c) Features clustered on calibration pattern, no keypoints on surrounding surfaces.



(d) Overexposed frame where tracking is lost due to sensor saturation.

Figure 12: Results on *V1_03_difficult*: map reconstructions (top) and frames illustrating the environmental limitations (bottom).